Analytics Buckets

Store large datasets for analytics and reporting.

This feature is in alpha

Expect rapid changes, limited features, and possible breaking updates. share feedback as we refine the experience and expand access.

Analytics buckets enable analytical workflows on large-scale datasets while keeping your primary database optimized for transactional operations.

Why Analytics buckets?

Postgres tables are purpose-built for transactional workloads with frequent inserts, updates, deletes, and low-latency queries. Analytical workloads have fundamentally different requirements:

- Processing large volumes of historical data
- Running complex queries and aggregations
- Minimizing storage costs
- Preventing analytical queries from impacting production traffic

Analytics buckets address these requirements using Apache Iceberg, an open-table format specifically designed for efficient management of large analytical datasets.

Ideal use cases

Analytics buckets are perfect for:

- **Data warehousing and business intelligence** - Build scalable data warehouses for BI tools
- **Historical data archiving** - Retain large volumes of historical data cost-effectively
- **Periodically refreshed analytics** - Maintain near real-time analytical views
- **Complex analytical queries** - Execute sophisticated aggregations and joins over large datasets

By separating transactional and analytical workloads, Supabase lets you build scalable analytics pipelines without compromising your primary Postgres performance.

Edit this page on GitHub

Need some help? Contact support

Latest product updates? See Changelog

Something's not right? Check system status

© Supabase Inc

—

Contributing

Author Styleguide

Open Source

SupaSquad

Privacy Settings

Creating Analytics Buckets

Set up your first analytics bucket using the SDK or dashboard.

This feature is in **Private Alpha**. API stability and backward compatibility are not guaranteed at this stage. Request access through this form.

Analytics buckets use Apache Iceberg, an open-table format for efficient management of large analytical datasets. You can interact with analytics buckets using tools such as PyIceberg, Apache Spark, or any client supporting the Iceberg REST Catalog API.

About replication

Analytics Buckets are still available, but managed replication into Analytics Buckets through Supabase ETL is no longer supported. If you need managed replication today, use Database Replication with **BigQuery**. If you want to use Analytics Buckets, bring your own ingestion pipeline.

Creating an Analytics bucket

You can create an analytics bucket using either the Supabase SDK or the Supabase Dashboard.

Using the Supabase SDK

JavaScript Python

```
import { createClient } from '@supabase/supabase-js'

const supabase = createClient('https://your-project.supabase.co', 'your-servi

const { data, error } = await supabase.storage.analytics.createBucket('analyt

if (error) {
  console.error('Failed to create analytics bucket:', error)
} else {
  console.log('Analytics bucket created:', data)
}
```

Using the Supabase Dashboard

1. Navigate to the **Storage** section in the Supabase Dashboard.
2. Click **Create Bucket**.
3. Enter a name for your bucket (e.g., `my-analytics-bucket`).
4. Select **Analytics Bucket** as the bucket type.
5. Click **Create**.

Load Supabase data in Python using dltHub

Build a Supabase-to-database pipeline in Python using dlt with AI Workbench support for Claude Code, Cursor, and Codex.

Last updated: Mar 6, 2026

In this guide, we'll set up a complete Supabase data pipeline from API credentials to your first data load in just 10 minutes. You'll end up with a fully declarative Python pipeline based on `dlt`'s REST API connector, like in the partial example code below:

EXAMPLE CODE

```
@dlt.source
def supabase_source(access_token=dlt.secrets.value):
    config: RESTAPIConfig = {
        "client": {
            "base_url": "https://api.supabase.com/v1",
            "auth": {
                "type": "bearer",
                "token": access_token,
            }
```

Why use dlt to generate Python pipelines?

- Accelerate pipeline development with AI-native context
- Debug pipelines, validate schemas and data with the integrated **Pipeline Dashboard**
- Build Python notebooks for end users of your data
- **Low maintenance** thanks to schema evolution with type inference, resilience and self-documenting REST API connectors. A shallow learning curve makes the pipeline easy to extend by any team member
- `dlt` is the tool of choice for Pythonic Iceberg Lakehouses, bringing mature data loading to Iceberg with or without catalogs

What you'll do

We'll show you how to generate a readable and easily maintainable Python script that fetches data from Supabase's API and loads it into Iceberg, DataFrames, files, or a database of your choice. Here are some of the endpoints you can load:

- **/v1/projects**: Endpoint for managing and retrieving project information.
- **/v1/organizations#**: Endpoint for managing and retrieving organization information.

You will then debug the Supabase pipeline using our Pipeline Dashboard tool to ensure it is copying the data correctly, before building a Notebook to explore your data and build reports.

Setup & steps to follow

💡 Before getting started, set up a virtual environment (instructions) and install the `dlt w`

```
uv venv && source .venv/bin/activate
uv pip install "dlt[workspace]"
```

Now you're ready to get started!

1. **Install the `dlt` AI Workbench**

   Configure the workbench for your coding assistant:

   ```
   dlt ai init --agent <your-agent> # <agent>: claude | cursor | codex
   ```

   Copy

   This installs project rules, a secrets management skill, appropriate ignore files, and configures the `dlt` MCP server for your agent.

   Learn more about the **dltHub AI Workbench** and setup details for each assistant →

2. **Install the `rest-api-pipeline` toolkit**

The **AI Workbench** provides different toolkits for each phase of the data engineering lifecycle. To start you need to install the `rest-api-pipeline` toolkit:

```
dlt ai toolkit rest-api-pipeline install
```

This loads different skills and contexts about `dlt` the agent uses to build the pipeline iteratively, efficiently, and safely. Importantly, it does not need to ask you for credentials directly. In `dlt`, API credentials are provided via a `secrets.toml` file (learn more about secrets management →), and the agent should use the MCP tools to see their shape and detect misconfigurations. It never needs to access the file directly.

Learn more about the `rest-api-pipeline` toolkit →

3. **Start LLM-assisted coding**

Here's a prompt to get you started:

PROMPT

Use /find-source to load data from the Supabase API into DuckDB.

The **AI Workbench** `rest-api-pipeline` toolkit takes over from here — it reads relevant API documentation, presents you with options for which endpoints to load, and then follows a structured workflow to scaffold, debug, and validate the pipeline step by step.

4. **View the result**

After the `rest-api-pipeline` workflow has finished, you will end up with a working REST API source with validated endpoints and a pipeline that writes data into a local dataset you have inspected and verified.

```
> python supabase_pipeline.py
Pipeline supabase load step completed in 0.26 seconds
1 load package(s) were loaded to destination duckdb and into dataset
```

```
The duckdb destination used duckdb:/supabase.duckdb location to stor
Load package 1749667187.541553 is LOADED and contains no failed jobs
```

By launching the **Pipeline Dashboard**, you can see various information about the pipeline and the loaded data

- Pipeline overview: State, load metrics
- Data's schema: tables, columns, types, hints
- You can query the data itself

```
dlt pipeline supabase_pipeline show
```

Copy

Next steps

You can go to the next phases of your data engineering journey by handing over to other toolkits of the **dltHub AI Workbench**:

- `data-exploration` — Build custom notebooks, charts, and dashboards for deeper analysis with marimo notebooks.
- `dlthub-runtime` — Deploy, schedule, and monitor your pipeline in production

```
dlt ai toolkit data-exploration install
dlt ai toolkit dlthub-runtime install
```

Copy

Or explore the following resources for more information:

- How to deploy a pipeline
- How to explore your data in marimo Notebooks
- How to query your data in Python with dataset

Was this page helpful?

Open in app ↗ Sign up Sign in

Medium Search Write

Building a Modular ETL Pipeline with Python, DBT, and Supabase

Charlesisaac Follow 2 min read · Oct 11, 2025

1

In the world of data engineering, simplicity and scalability often feel like opposing forces. But what if you could build a clean, modular ETL pipeline that's both easy to manage and ready for production? That's exactly what I set out to do — and in this article, I'll walk you through how I built it using Python, DBT, Supabase, and Docker.

🧱 The Goal

I wanted to create a lightweight ETL pipeline that:

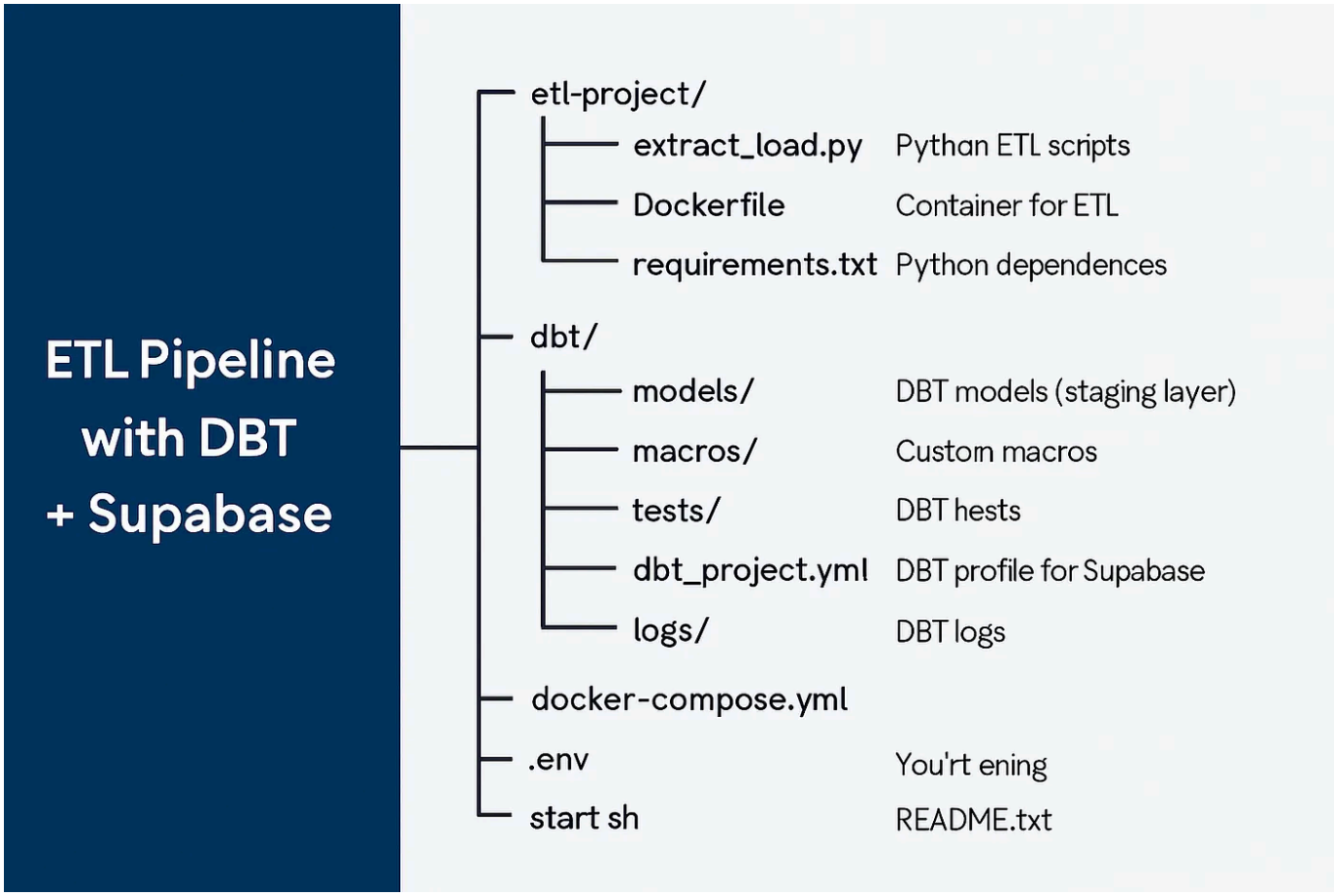
- Extracts raw data from a PostgreSQL database hosted on Supabase
- Transforms it into clean, validated staging models using DBT
- Runs inside Docker containers for portability
- Automates daily execution with cron
- Is version-controlled, documented, and ready for dashboard integration

⚙️ The Stack

- **Supabase:** PostgreSQL backend with a slick UI and easy setup
- **Python:** For writing the ETL logic
- **DBT:** For transforming and testing data
- **Docker:** To containerize the entire pipeline
- **Cron:** For scheduling daily runs

🧠 The Process

1. Designing the Structure



This modular layout keeps everything organized and scalable.

2. Writing the ETL Script

The `extract_load.py` script connects to Supabase using credentials from `.env`, creates staging tables, and copies data from raw tables. It's simple, readable, and easy to extend.

3. Modeling with DBT

Inside `dbt/dbt_project/models/`, I created SQL models like:

sql

```
-- staging_orders.sql
SELECT * FROM public.orders
```

Each model is materialized as a view or table, and tested using DBT's built-in schema tests (`not_null`, `unique`, `relationships`).

4. Containerizing with Docker

I wrote a `Dockerfile` for the ETL script and used `docker-compose.yml` to orchestrate both the ETL and DBT containers. This makes the pipeline portable and easy to deploy anywhere.

5. Automating with Cron

The `start.sh` script not only runs the pipeline manually, but also adds a cron job to execute it daily at midnight:

bash

```
0 0 * * * docker exec etl python etl/extract_load.py
```

This ensures the pipeline runs consistently without manual intervention.

✅ Final Touches

- Added a clean `README.md` with setup instructions, architecture, and usage
- Generated DBT documentation with `dbt docs generate`
- Version-controlled everything with Git
- Ready to connect to Metabase or Superset for dashboarding

🚀 What's Next

This pipeline is now ready for:

- Real-time data ingestion
- Incremental models
- CI/CD integration with GitHub Actions
- Visualization with BI tools

💬 Final Thoughts

Building this pipeline was a rewarding exercise in simplicity, modularity, and automation. Whether you're a solo data engineer or part of a growing analytics team, this setup offers a clean foundation for scalable data workflows.

Load GitHub Data to Supabase Using `dlt` in Python

TIP

We will be using the dlt PostgreSQL destination to connect to Supabase. You can get the connection string for your Supabase database as described in the Supabase Docs.

NEED HELP DEPLOYING THESE PIPELINES, OR FIGURING OUT HOW TO RUN THEM IN YOUR DATA STACK?

Join our Slack community or book a call with our support engineer Violetta.

This documentation explains how to load data from `GitHub` into `Supabase` using the open-source Python library `dlt`. The `dlt` library allows you to extract data on issues, pull requests, or events from any `GitHub` repository and load it into a destination of your choice, such as `Supabase`. `Supabase` is an open-source alternative to Firebase, offering a Postgres database, authentication, instant APIs, edge functions, real-time subscriptions, storage, and vector embeddings. For more detailed information about the `GitHub` API, visit GitHub's documentation.

`dlt` Key Features

- **Automated maintenance**: With schema inference and evolution and alerts, and with short declarative code, maintenance becomes simple. Learn more
- **Run it where Python runs**: On Airflow, serverless functions, notebooks. No external APIs, backends or containers, scales on micro and large infra alike. Learn more
- **User-friendly interface**: A declarative interface that removes knowledge obstacles for beginners while empowering senior professionals. Learn more
- **Detailed tutorial available**: A step-by-step guide to building a pipeline that loads data from the GitHub API into DuckDB. Follow the tutorial

Get GPT-4 Help

- **Community support**: Join the `dlt` community on Slack for help and support, and give the library a ⭐ on GitHub. Join Slack

Getting started with your pipeline locally

0. Prerequisites

`dlt` requires Python 3.8 or higher. Additionally, you need to have the `pip` package manager installed, and we recommend using a virtual environment to manage your dependencies. You can learn more about preparing your computer for dlt in our installation reference.

1. Install dlt

First you need to install the `dlt` library with the correct extras for `Supabase`:

```
pip install "dlt[postgres]"
```

The `dlt` cli has a useful command to get you started with any combination of source and destination. For this example, we want to load data from `GitHub` to `Supabase`. You can run the following commands to create a starting point for loading data from `GitHub` to `Supabase`:

```
# create a new directory
mkdir github_pipeline
cd github_pipeline
# initialize a new pipeline with your source and destination
dlt init github postgres
# install the required dependencies
pip install -r requirements.txt
```

The last command will install the required dependencies for your pipeline. The dependencies are listed in the `requirements.txt`:

```
dlt[postgres]>=0.3.25
```

You now have the following folder structure in your project:

```
github_pipeline/
├── .dlt/
│   ├── config.toml          # configs for your pipeline
│   └── secrets.toml         # secrets for your pipeline
├── github/                   # folder with source specific files
│   └── ...
├── github_pipeline.py        # your main pipeline script
├── requirements.txt         # dependencies for your pipeline
└── .gitignore               # ignore files for git (not required)
```

2. Configuring your source and destination credentials

The `dlt` cli will have created a `.dlt` directory in your project folder. This directory contains a `config.toml` file and a `secrets.toml` file that you can use to configure your pipeline. The automatically created version of these files look like this:

generated config.toml

```
# put your configuration values here

[runtime]
log_level="WARNING"  # the system log level of dlt
# use the dlthub_telemetry setting to enable/disable anonymous usage data
reporting, see https://dlthub.com/docs/telemetry
dlthub_telemetry = true
```

generated secrets.toml

```
# put your secret values and credentials here. do not share this file and do
not push it to github

[sources.github]
access_token = "access_token" # please set me up!

[destination.postgres]
dataset_name = "dataset_name" # please set me up!

[destination.postgres.credentials]
database = "database" # please set me up!
password = "password" # please set me up!
username = "username" # please set me up!
```

```
host = "host" # please set me up!
port = 5432
connect_timeout = 15
```

2.1. Adjust the generated code to your usecase

> **FURTHER HELP SETTING UP YOUR SOURCE AND DESTINATIONS**
>
> - Read more about setting up the `GitHub` source in our docs.
> - Read more about setting up the `Supabase` destination in our docs.

3. Running your pipeline for the first time

The `dlt` cli has also created a main pipeline script for you at `github_pipeline.py`, as well as a folder `github` that contains additional python files for your source. These files are your local copies which you can modify to fit your needs. In some cases you may find that you only need to do small changes to your pipelines or add some configurations, in other cases these files can serve as a working starting point for your code, but will need to be adjusted to do what you need them to do.

The main pipeline script will look something like this:

```
import dlt

from github import github_reactions, github_repo_events


def load_duckdb_repo_reactions_issues_only() -> None:
    """Loads issues, their comments and reactions for duckdb"""
    pipeline = dlt.pipeline(
        "github_reactions",
        destination='postgres',
        dataset_name="duckdb_issues",
        full_refresh=True,
    )
    # get only 100 items (for issues and pull request)
    data = github_reactions(
        "duckdb", "duckdb", items_per_page=100, max_items=100
    ).with_resources("issues")
```

```python
    print(pipeline.run(data))


def load_airflow_events() -> None:
    """Loads airflow events. Shows incremental loading. Forces anonymous
access token"""
    pipeline = dlt.pipeline(
        "github_events", destination='postgres', dataset_name="airflow_events"
    )
    data = github_repo_events("apache", "airflow", access_token="")
    print(pipeline.run(data))
    # if you uncomment this, it does not load the same events again
    # data = github_repo_events("apache", "airflow", access_token="")
    # print(pipeline.run(data))


def load_dlthub_dlt_all_data() -> None:
    """Loads all issues, pull requests and comments for dlthub dlt repo"""
    pipeline = dlt.pipeline(
        "github_reactions",
        destination='postgres',
        dataset_name="dlthub_reactions",
        full_refresh=True,
    )
    data = github_reactions("dlt-hub", "dlt")
    print(pipeline.run(data))


if __name__ == "__main__":
    load_duckdb_repo_reactions_issues_only()
    load_airflow_events()
    load_dlthub_dlt_all_data()
```

Provided you have set up your credentials, you can run your pipeline like a regular python script with the following command:

```
python github_pipeline.py
```

4. Inspecting your load result

You can now inspect the state of your pipeline with the `dlt` cli:

```
dlt pipeline github_events info
```

You can also use streamlit to inspect the contents of your `Supabase` destination for this:

```
# install streamlit
pip install streamlit
# run the streamlit app for your pipeline with the dlt cli:
dlt pipeline github_events show
```

5. Next steps to get your pipeline running in production

One of the beauties of `dlt` is, that we are just a plain Python library, so you can run your pipeline in any environment that supports Python >= 3.8. We have a couple of helpers and guides in our docs to get you there:

The Deploy section will show you how to deploy your pipeline to

- **Deploy with Github Actions**: Learn how to deploy a pipeline using Github Actions.
- **Deploy with Airflow**: Follow the steps to deploy a pipeline with Airflow and Google Composer.
- **Deploy with Google Cloud Functions**: Discover how to deploy your pipeline using Google Cloud Functions.
- **Explore other deployment options**: Check out other methods to deploy your pipeline here.

The running in production section will teach you about:

- **How to Monitor your pipeline**: Learn how to effectively monitor your `dlt` pipeline to ensure smooth operations and quickly identify any issues. Read more
- **Set up alerts**: Configure alerts to get notified about important events and potential issues in your `dlt` pipeline. Read more
- **Set up tracing**: Implement tracing to gain detailed insights into the execution of your `dlt` pipeline, helping you to diagnose and resolve issues efficiently. Read more

Available Sources and Resources

For this verified source the following sources and resources are available

Source `github_repo_events`

"GitHub repo events source provides data about activities and interactions within a repository."

Resource Name	Write Disposition	Description
`repo_events`	`append`	Retrieves all the repository events associated with the GitHub repository. This includes information about the actor (user who triggered the event), organization, payload (specific details about the event), and the repository itself.

Additional pipeline guides

- Load data from Klarna to ClickHouse in python with dlt
- Load data from Braze to Databricks in python with dlt
- Load data from MySQL to MotherDuck in python with dlt
- Load data from Bitbucket to Redshift in python with dlt
- Load data from Looker to ClickHouse in python with dlt
- Load data from Chess.com to Neon Serverless Postgres in python with dlt
- Load data from Attio to Supabase in python with dlt
- Load data from Adobe Commerce (Magento) to AWS Athena in python with dlt
- Load data from IBM Db2 to Databricks in python with dlt
- Load data from Pinterest to Snowflake in python with dlt

Edit this page

Last updated on **Aug 7, 2024** by **Zaeem Athar**

Back to integrations

Kafka Supabase

Kafka and Supabase integration

Save yourself the work of writing custom integrations for Kafka and Supabase and use n8n instead. Build adaptable and scalable Development, workflows that work with your technology stack. All within a building experience you will love.

Get Started

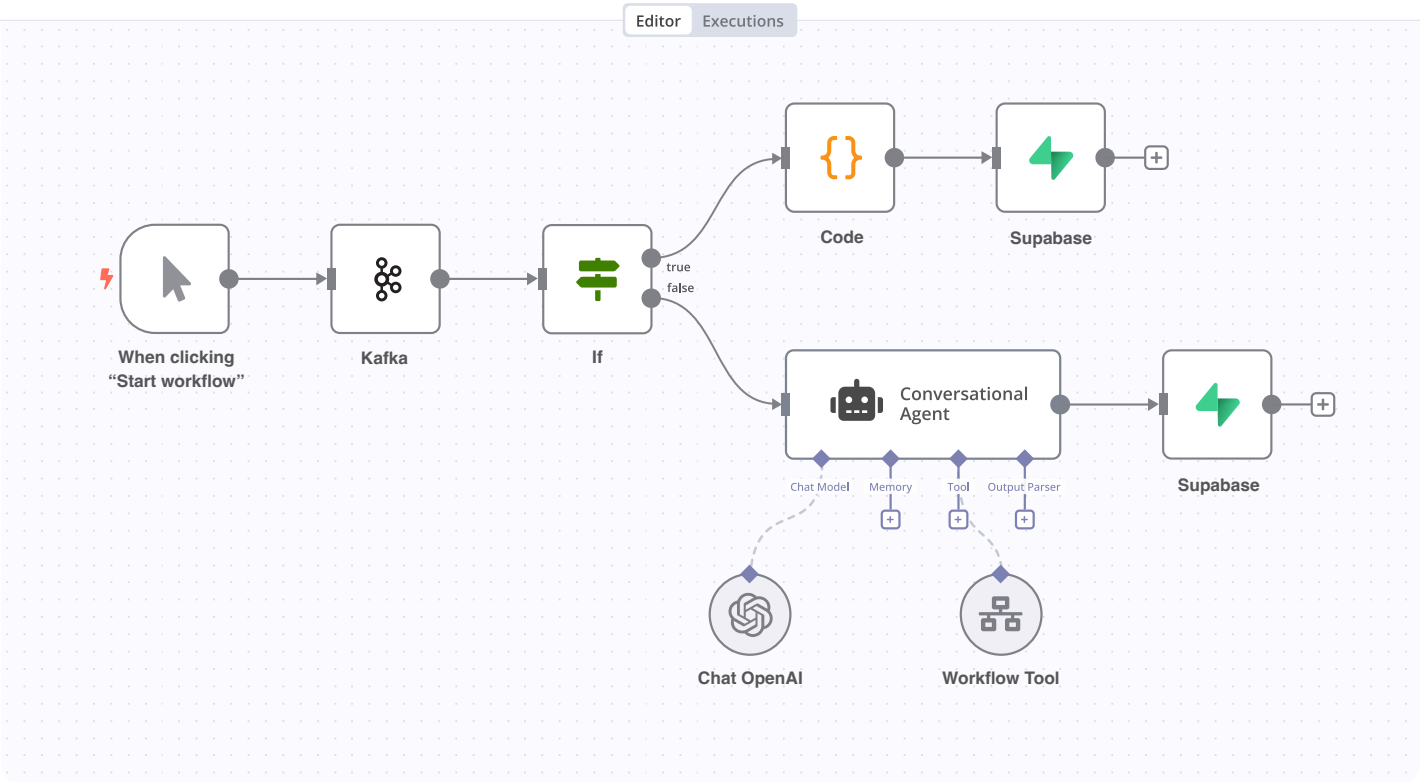
See n8n in action

How to connect Kafka and Supabase

Step 1 Step 2 Step 3 Step 4 Step 5

Customize and extend your Kafka and Supabase integration

Use n8n's core nodes such as If, Split Out, Merge, and others to transform and manipulate data. Write custom JavaScript or Python in the Code node and run it as a step in your workflow. Connect Kafka and Supabase with any



Build your own Kafka and Supabase integration

Create custom Kafka and Supabase workflows by choosing triggers and actions. Nodes come with global operations and settings, as well as app-specific parameters that can be configured. You can also use the HTTP Request node to query data from any app or service with a REST API.

Supabase supported actions

Row

Create

Create a new row

Connect a Supabase Database to Kestra

Table of Contents

Connect your Supabase Database to your workflows using the PostgreSQL plugin.

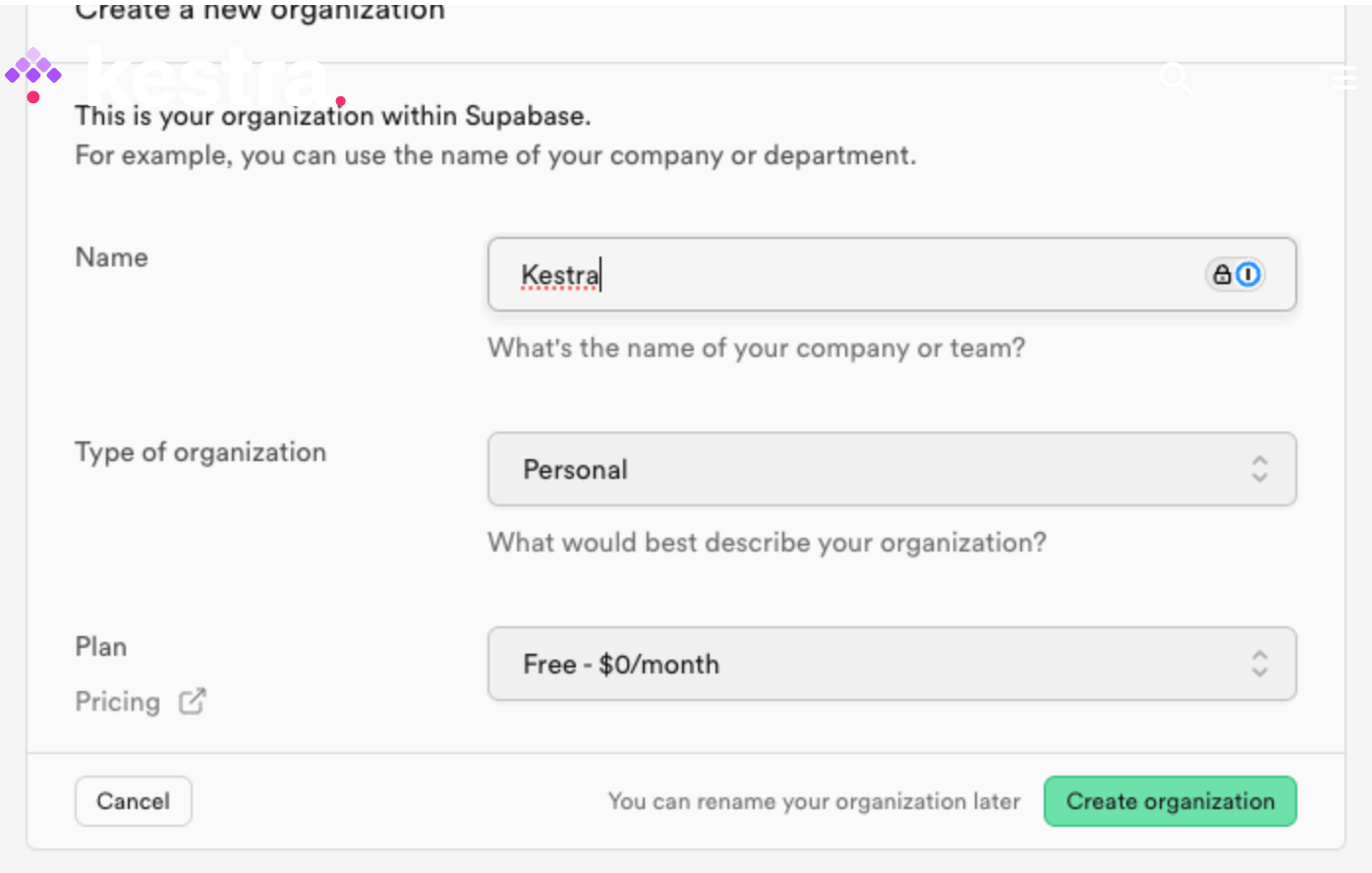
> There is a dedicated Supabase plugin to replace these steps.

Supabase is an open-source Backend-as-a-service (BaaS) platform that helps developers build applications faster and more efficiently. They provide a number of services, including hosted PostgreSQL databases, which can be used within Flows in Kestra.

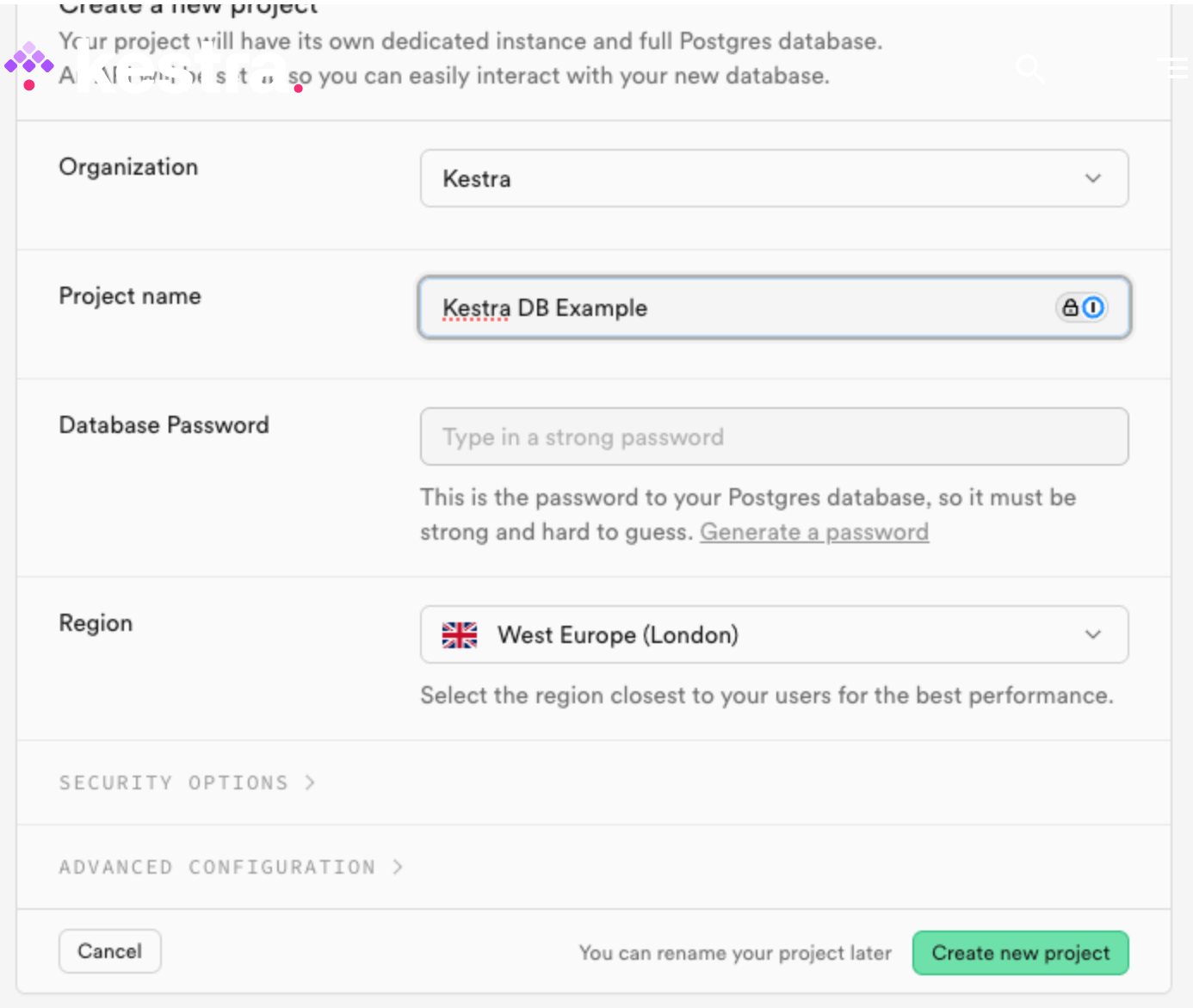
To get started, make sure you have a Supabase account set up and an installation of Kestra running.

Setting up a Database in Supabase

Once you've logged into Supabase, you'll need to set up an organization where you will create projects to access resources such as a database.



Once your organization is created, you'll be prompted to create a new project. You will need to set a password for this project which we will use later to authenticate with the database in Kestra.



Once your project is created, you will now be able to access resources in Supabase. Head to the menu on the left side and select **Database**. You will be prompted to create a new table in your database, as well as configure any columns you want to use. We can leave the columns blank for now and modify these later once we know what data we want to copy into our database.

Name

kestra_example

Description

Optional

Enable Row Level Security (RLS) Recommended

Restrict access to your table by enabling RLS and writing Postgres policies.

Policies are required to query data

You need to create an access policy before you can query data from this table. Without a policy, querying this table will return an empty array of results. You can create policies after saving this table.

Documentation

Enable Realtime

Broadcast changes on this table to authorized subscribers

Columns

About data types | Import data from CSV

Name	Type	Default Value	Primary
id	# int8	NULL	☑
created_at	timestam	now()	☐

Add column

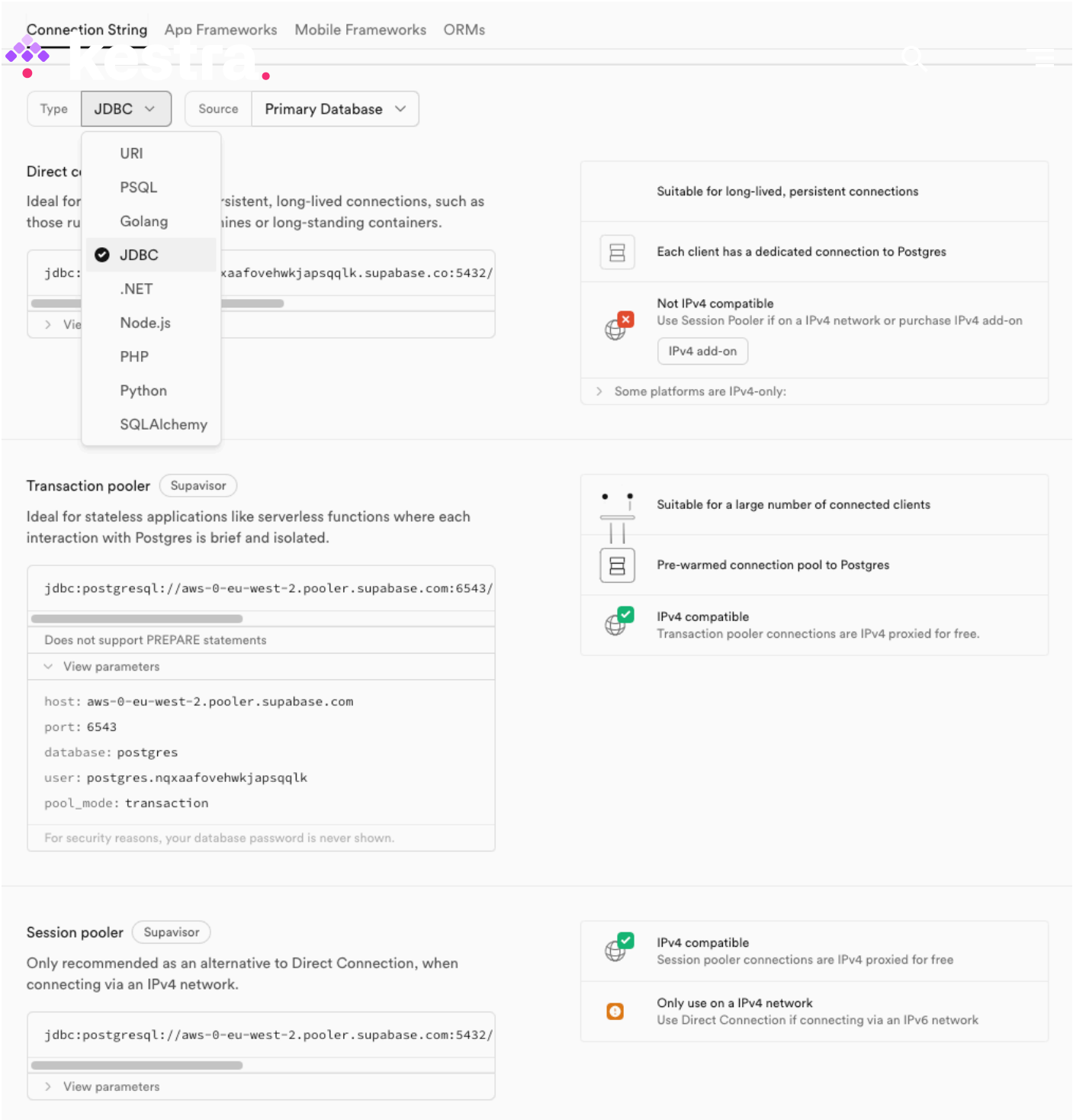
Foreign keys

Add foreign key relation

Connecting Supabase to Kestra

Now that we have a database set up in Supabase, we can move into Kestra to set up our connection. While there's no official Supabase plugin, we can connect using the PostgreSQL plugin, which supports a number of tasks such as `Query`, `CopyIn`, and `CopyOut`.

Inside of Supabase, select the **Connect** button at the top to get information about our databases connection. Select **Type** and change this JDBC. This will give us 3 ways of connecting with a Connection String. As we're only connecting to the database when our workflow runs, the Transaction pooler is a good option to use.



To connect, we can copy the URL provided for the Transaction pooler and replace `[YOUR-PASSWORD]` with the password set earlier. To prevent exposing the password in our flow, store it as a secret.

By using Plugin Defaults, we can configure our connection to Supabase once for all tasks in our flow rather than individually for each task.

Once configured, our connection in Kestra will look like the example below:

```
  - forced: true
    type: io.kestra.plugin.jdbc.postgresql
    values:
      url: "jdbc:postgresql://aws-0-eu-west-2.pooler.supabase.com:6543/postgres?
      user=postgres.nqxaafovehwkjapsqqlk&password={{ secret('SUPABASE_PASSWORD')
      }}"
```

You can also use the `username` and `password` properties rather than combining it all into the `url` property:

```yaml
pluginDefaults:
  - forced: true
    type: io.kestra.plugin.jdbc.postgresql
    values:
      url: "jdbc:postgresql://aws-0-eu-west-
      2.pooler.supabase.com:6543/postgres"
      username: "postgres.nqxaafovehwkjapsqqlk"
      password: "{{ secret('SUPABASE_PASSWORD') }}"
```

Copying a CSV File into Supabase DB in a Flow

Using this example CSV , we can copy the data into our table directly from Kestra. You can either set up the columns directly in Supabase or add a task in Kestra to add them automatically like this:

```yaml
id: supabase_db_add_columns
namespace: company.team

tasks:
  - id: create_columns
    type: io.kestra.plugin.jdbc.postgresql.Queries
    sql: |
      ALTER TABLE kestra_example
      ADD COLUMN order_id int,
```

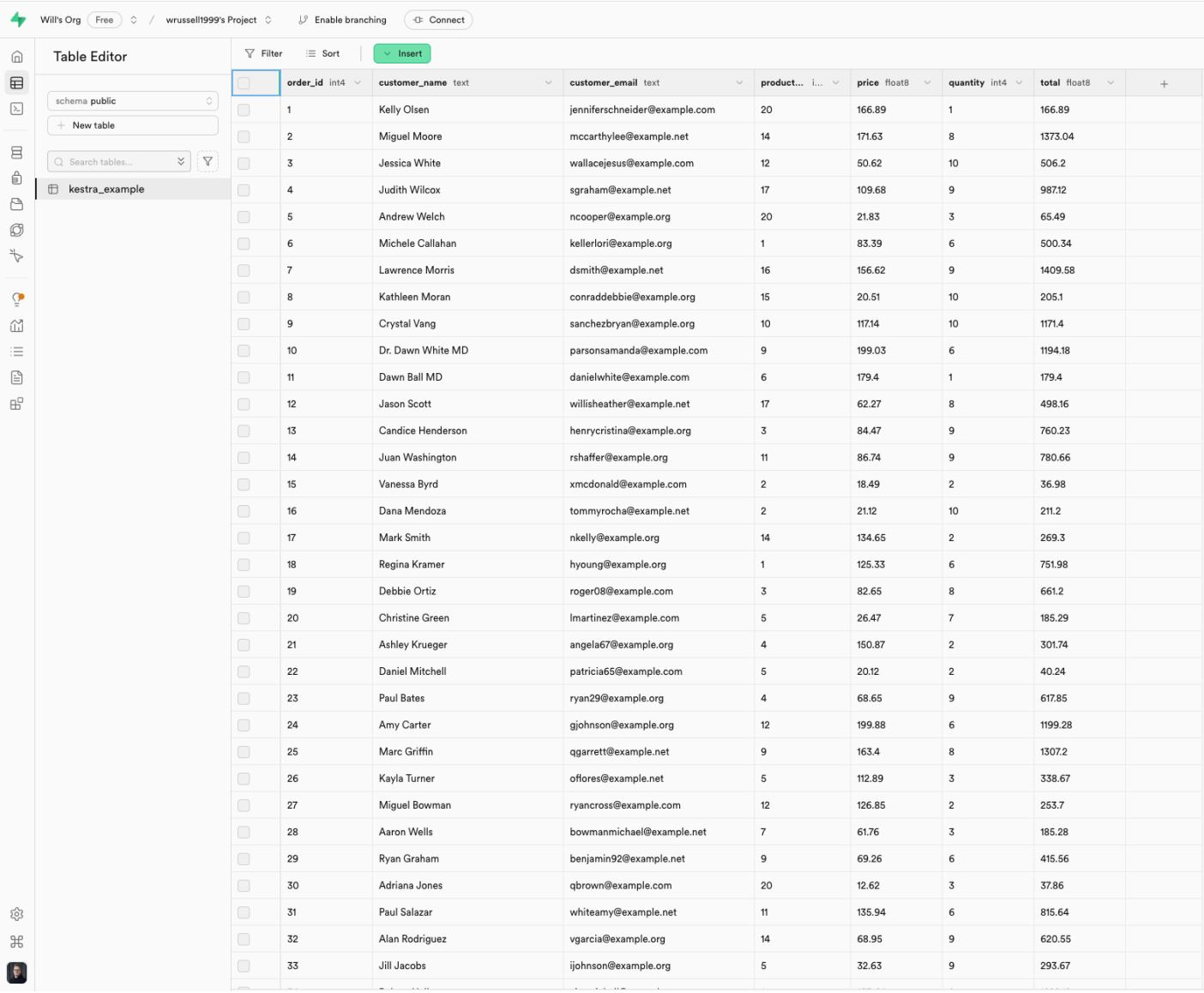
```
      ADD COLUMN product_id int,
      ADD COLUMN price double precision,
      ADD COLUMN quantity int,
      ADD COLUMN total double precision;

pluginDefaults:
  - forced: true
    type: io.kestra.plugin.jdbc.postgresql
    values:
      url: "jdbc:postgresql://aws-0-eu-west-2.pooler.supabase.com:6543/postgres?
      user=postgres.nqxaafovehwkjapsqqlk&password={{ secret('SUPABASE_PASSWORD')
      }}"
```

Once your columns are configured, you can use the CopyIn task combined with the HTTP Download task to download the CSV file and copy it directly into our database. As we set up the database connection with our Plugin Defaults, the CopyIn task will connect directly and copy the CSV file into the database.

```yaml
id: supabase_db_copyin
namespace: company.team

tasks:
  - id: download
    type: io.kestra.plugin.core.http.Download
    uri: https://huggingface.co/datasets/kestra/datasets/raw/main/csv/orders.csv

  - id: copy_in
    type: io.kestra.plugin.jdbc.postgresql.CopyIn
    table: "kestra_example"
    from: "{{ outputs.download.uri }}"
    header: true
    columns:
    [order_id,customer_name,customer_email,product_id,price,quantity,total]
    delimiter: ","

pluginDefaults:
  - forced: true
    type: io.kestra.plugin.jdbc.postgresql
    values:
```

```
}}"
```

Once this flow completes, we can view the contents of our database in Supabase:



order_id int4	customer_name text	customer_email text	product... i...	price float8	quantity int4	total float8
1	Kelly Olsen	jenniferschneider@example.com	20	166.89	1	166.89
2	Miguel Moore	mccarthylee@example.net	14	171.63	8	1373.04
3	Jessica White	wallacejesus@example.com	12	50.62	10	506.2
4	Judith Wilcox	sgraham@example.net	17	109.68	9	987.12
5	Andrew Welch	ncooper@example.org	20	21.83	3	65.49
6	Michele Callahan	kellerlori@example.org	1	83.39	6	500.34
7	Lawrence Morris	dsmith@example.net	16	156.62	9	1409.58
8	Kathleen Moran	conraddebbie@example.org	15	20.51	10	205.1
9	Crystal Vang	sanchezbryan@example.org	10	117.14	10	1171.4
10	Dr. Dawn White MD	parsonsamanda@example.com	9	199.03	6	1194.18
11	Dawn Ball MD	danielwhite@example.com	6	179.4	1	179.4
12	Jason Scott	willisheather@example.net	17	62.27	8	498.16
13	Candice Henderson	henrycristina@example.org	3	84.47	9	760.23
14	Juan Washington	rshaffer@example.org	11	86.74	9	780.66
15	Vanessa Byrd	xmcdonald@example.com	2	18.49	2	36.98
16	Dana Mendoza	tommyrocha@example.net	2	21.12	10	211.2
17	Mark Smith	nkelly@example.org	14	134.65	2	269.3
18	Regina Kramer	hyoung@example.org	1	125.33	6	751.98
19	Debbie Ortiz	roger08@example.com	3	82.65	8	661.2
20	Christine Green	lmartinez@example.com	5	26.47	7	185.29
21	Ashley Krueger	angela67@example.org	4	150.87	2	301.74
22	Daniel Mitchell	patricia65@example.com	5	20.12	2	40.24
23	Paul Bates	ryan29@example.org	4	68.65	9	617.85
24	Amy Carter	gjohnson@example.org	12	199.88	6	1199.28
25	Marc Griffin	qgarrett@example.net	9	163.4	8	1307.2
26	Kayla Turner	oflores@example.net	5	112.89	3	338.67
27	Miguel Bowman	ryancross@example.com	12	126.85	2	253.7
28	Aaron Wells	bowmanmichael@example.net	7	61.76	3	185.28
29	Ryan Graham	benjamin92@example.net	9	69.26	6	415.56
30	Adriana Jones	qbrown@example.com	20	12.62	3	37.86
31	Paul Salazar	whiteamy@example.net	11	135.94	6	815.64
32	Alan Rodriguez	vgarcia@example.org	14	68.95	9	620.55
33	Jill Jacobs	ijohnson@example.org	5	32.63	9	293.67

Previous

Sync Flows from a Git Repository

Was this page helpful?

Yes

No

Loading Data from `Rest API` to `Supabase` in Python with `dlt`

ABOUT OUR REST_API VERIFIED SOURCE

This example demonstrates how to use the rest_api to retrieve data from the GitHub Rest API, but will work with any HTTP Rest API. Please read:

- The rest_api docs to learn how to configure this verified source
- The OpenAPI generator docs to learn how to automatically configure a dlt rest_api source from an OpenAPI spec
- Our cool google colab example demonstrating the generator and the rest_api source

TIP

We will be using the dlt PostgreSQL destination to connect to Supabase. You can get the connection string for your Supabase database as described in the Supabase Docs.

NEED HELP DEPLOYING THESE PIPELINES, OR FIGURING OUT HOW TO RUN THEM IN YOUR DATA STACK?

Join our Slack community or book a call with our support engineer Violetta.

The `rest_api` verified source supports getting data from any HTTP `Rest API` to `Supabase`. `Supabase` is an open-source Firebase alternative that offers a Postgres database, Authentication, instant APIs, Edge Functions, Realtime subscriptions, Storage, and Vector embeddings. Using the open-source Python library `dlt`, you can easily integrate and manage your data pipeline. For further information on the `rest_api` source, visit this link.

`dlt` Key Features

- **Automated Maintenance**: With schema inference, evolution, and alerts, `dlt` simplifies maintenance. Learn more
- **Run Anywhere Python Runs**: Deploy on Airflow, serverless functions, notebooks, and more. Find out how
- **User-Friendly Interface**: Declarative interface that lowers knowledge barriers for beginners and empowers professionals. Explore the interface
- **Securely Handle Secrets**: Securely store sensitive information like access tokens in the `.dlt` folder. Handle secrets securely
- **Incremental Loading**: Load only new data and deduplicate existing data with incremental loading. Understand incremental loading

Getting started with your pipeline locally

0. Prerequisites

`dlt` requires Python 3.8 or higher. Additionally, you need to have the `pip` package manager installed, and we recommend using a virtual environment to manage your dependencies. You can learn more about preparing your computer for dlt in our installation reference.

1. Install dlt

First you need to install the `dlt` library with the correct extras for `Supabase`:

```
pip install "dlt[postgres]"
```

The `dlt` cli has a useful command to get you started with any combination of source and destination. For this example, we want to load data from `Rest API` to `Supabase`. You can run the following commands to create a starting point for loading data from `Rest API` to `Supabase`:

```
# create a new directory
mkdir rest_api_pipeline
cd rest_api_pipeline
# initialize a new pipeline with your source and destination
dlt init rest_api postgres
```

```
# install the required dependencies
pip install -r requirements.txt
```

The last command will install the required dependencies for your pipeline. The dependencies are listed in the `requirements.txt`:

```
dlt[postgres]>=0.4.11
```

You now have the following folder structure in your project:

```
rest_api_pipeline/
├── .dlt/
│   ├── config.toml          # configs for your pipeline
│   └── secrets.toml         # secrets for your pipeline
├── rest_api/                   # folder with source specific files
│   └── ...
├── rest_api_pipeline.py        # your main pipeline script
├── requirements.txt         # dependencies for your pipeline
└── .gitignore               # ignore files for git (not required)
```

2. Configuring your source and destination credentials

The `dlt` cli will have created a `.dlt` directory in your project folder. This directory contains a `config.toml` file and a `secrets.toml` file that you can use to configure your pipeline. The automatically created version of these files look like this:

generated config.toml

```
# put your configuration values here

[runtime]
log_level="WARNING"  # the system log level of dlt
# use the dlthub_telemetry setting to enable/disable anonymous usage data
reporting, see https://dlthub.com/docs/telemetry
dlthub_telemetry = true
```

generated secrets.toml

```
# put your secret values and credentials here. do not share this file and do
not push it to github

[sources.rest_api]
github_token = "github_token" # please set me up!

[destination.postgres]
dataset_name = "dataset_name" # please set me up!

[destination.postgres.credentials]
database = "database" # please set me up!
password = "password" # please set me up!
username = "username" # please set me up!
host = "host" # please set me up!
port = 5432
connect_timeout = 15
```

2.1. Adjust the generated code to your usecase

> **FURTHER HELP SETTING UP YOUR SOURCE AND DESTINATIONS**
>
> - Read more about setting up the `Rest API` source in our docs.
> - Read more about setting up the `Supabase` destination in our docs.

3. Running your pipeline for the first time

The `dlt` cli has also created a main pipeline script for you at `rest_api_pipeline.py`, as well as a folder `rest_api` that contains additional python files for your source. These files are your local copies which you can modify to fit your needs. In some cases you may find that you only need to do small changes to your pipelines or add some configurations, in other cases these files can serve as a working starting point for your code, but will need to be adjusted to do what you need them to do.

The main pipeline script will look something like this:

```
from typing import Any

import dlt
```

```python
from rest_api import (
    RESTAPIConfig,
    check_connection,
    rest_api_source,
    rest_api_resources,
)


@dlt.source
def github_source(github_token: str = dlt.secrets.value) -> Any:
    # Create a REST API configuration for the GitHub API
    # Use RESTAPIConfig to get autocompletion and type checking
    config: RESTAPIConfig = {
        "client": {
            "base_url": "https://api.github.com/repos/dlt-hub/dlt/",
            "auth": {
                "type": "bearer",
                "token": github_token,
            },
        },
        # The default configuration for all resources and their endpoints
        "resource_defaults": {
            "primary_key": "id",
            "write_disposition": "merge",
            "endpoint": {
                "params": {
                    "per_page": 100,
                },
            },
        },
        "resources": [
            # This is a simple resource definition,
            # that uses the endpoint path as a resource name:
            # "pulls",
            # Alternatively, you can define the endpoint as a dictionary
            # {
            #     "name": "pulls", # <- Name of the resource
            #     "endpoint": "pulls",  # <- This is the endpoint path
            # }
            # Or use a more detailed configuration:
            {
                "name": "issues",
                "endpoint": {
                    "path": "issues",
                    # Query parameters for the endpoint
                    "params": {
                        "sort": "updated",
```

```
                        "direction": "desc",
                        "state": "open",
                        # Define `since` as a special parameter
                        # to incrementally load data from the API.
                        # This works by getting the updated_at value
                        # from the previous response data and using this value
                        # for the `since` query parameter in the next request.
                        "since": {
                            "type": "incremental",
                            "cursor_path": "updated_at",
                            "initial_value": "2024-01-25T11:21:28Z",
                        },
                    },
                },
            },
            # The following is an example of a resource that uses
            # a parent resource (`issues`) to get the `issue_number`
            # and include it in the endpoint path:
            {
                "name": "issue_comments",
                "endpoint": {
                    # The placeholder {issue_number} will be resolved
                    # from the parent resource
                    "path": "issues/{issue_number}/comments",
                    "params": {
                        # The value of `issue_number` will be taken
                        # from the `number` field in the `issues` resource
                        "issue_number": {
                            "type": "resolve",
                            "resource": "issues",
                            "field": "number",
                        }
                    },
                },
                # Include data from `id` field of the parent resource
                # in the child data. The field name in the child data
                # will be called `_issues_id` (_{resource_name}_{field_name})
                "include_from_parent": ["id"],
            },
        ],
    }

    yield from rest_api_resources(config)


def load_github() -> None:
    pipeline = dlt.pipeline(
```

```
        pipeline_name="rest_api_github",
        destination='postgres',
        dataset_name="rest_api_data",
    )

    load_info = pipeline.run(github_source())
    print(load_info)


def load_pokemon() -> None:
    pipeline = dlt.pipeline(
        pipeline_name="rest_api_pokemon",
        destination='postgres',
        dataset_name="rest_api_data",
    )

    pokemon_source = rest_api_source(
        {
            "client": {
                "base_url": "https://pokeapi.co/api/v2/",
                # If you leave out the paginator, it will be inferred from the
API:
                # paginator: "json_response",
            },
            "resource_defaults": {
                "endpoint": {
                    "params": {
                        "limit": 1000,
                    },
                },
            },
            "resources": [
                "pokemon",
                "berry",
                "location",
            ],
        }
    )

    def check_network_and_authentication() -> None:
        (can_connect, error_msg) = check_connection(
            pokemon_source,
            "not_existing_endpoint",
        )
        if not can_connect:
            pass  # do something with the error message
```

```
    check_network_and_authentication()

    load_info = pipeline.run(pokemon_source)
    print(load_info)


if __name__ == "__main__":
    load_github()
    load_pokemon()
```

Provided you have set up your credentials, you can run your pipeline like a regular python script with the following command:

```
python rest_api_pipeline.py
```

4. Inspecting your load result

You can now inspect the state of your pipeline with the `dlt` cli:

```
dlt pipeline rest_api_github info
```

You can also use streamlit to inspect the contents of your `Supabase` destination for this:

```
# install streamlit
pip install streamlit
# run the streamlit app for your pipeline with the dlt cli:
dlt pipeline rest_api_github show
```

5. Next steps to get your pipeline running in production

One of the beauties of `dlt` is, that we are just a plain Python library, so you can run your pipeline in any environment that supports Python >= 3.8. We have a couple of helpers and guides in our docs to get you there:

The Deploy section will show you how to deploy your pipeline to

- **Deploy with GitHub Actions**: Learn how to set up and deploy your `dlt` pipeline using GitHub Actions for automated workflows. Read more
- **Deploy with Airflow and Google Composer**: Follow this guide to deploy your `dlt` pipeline using Airflow and Google Composer for managed workflow orchestration. Read more
- **Deploy with Google Cloud Functions**: Discover how to deploy your `dlt` pipeline using Google Cloud Functions for serverless execution. Read more
- **Explore more deployment options**: Check out other methods to deploy your `dlt` pipeline, including various cloud services and orchestration tools. Read more

The running in production section will teach you about:

- **How to Monitor your pipeline**: Learn how to effectively monitor your `dlt` pipeline in production to ensure smooth and reliable operation. How to Monitor your pipeline
- **Set up alerts**: Set up alerts to get notified of any issues or important events occurring in your `dlt` pipeline. Set up alerts
- **And set up tracing**: Implement tracing to gain insights into the performance and behavior of your `dlt` pipeline. And set up tracing

Available Sources and Resources

For this verified source the following sources and resources are available

Source `github_source`

"Rest API Source for GitHub, providing detailed data on issues and related comments."

Resource Name	Write Disposition	Description
`issue_comments`	`merge`	Contains information about the issue comments including the author, body of the comment, created date, and user details among other data.

Resource Name	Write Disposition	Description
`issues`	`merge`	Contains information about the issues including the assignee details, author, body of the issue, comments, created date, and user details among other data.

Additional pipeline guides

- Load data from Looker to AlloyDB in python with dlt
- Load data from Salesforce to EDB BigAnimal in python with dlt
- Load data from Aladtec to Snowflake in python with dlt
- Load data from Trello to CockroachDB in python with dlt
- Load data from Box Platform API to ClickHouse in python with dlt
- Load data from Salesforce to Timescale in python with dlt
- Load data from Salesforce to Microsoft SQL Server in python with dlt
- Load data from Adobe Commerce (Magento) to Dremio in python with dlt
- Load data from Jira to Microsoft SQL Server in python with dlt
- Load data from The Local Filesystem to AWS S3 in python with dlt

Edit this page

Last updated on **Aug 7, 2024** by **Zaeem Athar**

AWS S3

You can enable the AWS S3 wrapper right from the Supabase dashboard.

Open wrapper in dashboard

AWS S3 is an object storage service offering industry-leading scalability, data availability, security, and performance. It is read-only and supports below file formats:

The S3 Wrapper allows you to read data of below formats from S3 within your Postgres database.

1. CSV - with or without header line
2. JSON Lines
3. Parquet

The S3 Wrapper also supports below compression algorithms:

1. gzip
2. bzip2
3. xz
4. zlib

Note for CSV and JSONL files: currently all columns in S3 files must be defined in the foreign table and their types must be `text` type.

Note for Parquet files: the whole Parquet file will be loaded into local memory if it is compressed, so keep the file size as small as possible.

Preparation

Before you can query S3, you need to enable the Wrappers extension and store your credentials in Postgres.

Enable Wrappers

Make sure the `wrappers` extension is installed on your database:

```
create extension if not exists wrappers with schema extensions;
```

Enable the S3 Wrapper

Enable the `s3_wrapper` FDW:

```
create foreign data wrapper s3_wrapper
  handler s3_fdw_handler
  validator s3_fdw_validator;
```

Store your credentials (optional)

By default, Postgres stores FDW credentials inside `pg_catalog.pg_foreign_server` in plain text. Anyone with access to this table will be able to view these credentials. Wrappers is designed to work with Vault, which provides an additional level of security for storing credentials. We recommend using Vault to store your credentials.

```
-- Save your AWS credentials in Vault and retrieve the created `s3_access_key
```

```
select vault.create_secret(
  '<access key id>',
  's3_access_key_id',
  'AWS access key for Wrappers'
);
select vault.create_secret(
  '<secret access key>'
  's3_secret_access_key',
  'AWS secret access key for Wrappers'
```

Database

Connecting to S3

We need to provide Postgres with the credentials to connect to S3, and any additional options. We can do this using the `create server` command:

With Vault | Without Vault

```
create server s3_server
  foreign data wrapper s3_wrapper
  options (
    vault_access_key_id '<your s3_access_key_id from above>',
    vault_secret_access_key '<your s3_secret_access_key from above>',
    aws_region 'us-east-1'
  );
```

The full list of options are below:

- `aws_access_key_id` (required) - Your access key
- `aws_secret_access_key` (required) - Your secret key
- `aws_region` (required) - The region of your bucket (if providing an endpoint URL with a region in it, make sure that they are the same)
- `endpoint_url` (optional) - An optional URL to allow connection to S3-compliant providers (i.e. Wasabi, Cloudflare R2, Backblaze B2, DigitalOcean Spaces)

`path_style_url` (optional) - Whether to use path-style URL access. This is required by some S3-compliant providers. `true` or `false`, default is `false`.

Required S3 permissions

Below S3 permissions are needed:

s3:GetObject

s3:GetObjectAttributes

If the bucket is versioned, we also need:

s3:GetObjectVersion

s3:GetObjectVersionAttributes

Connecting to S3-compliant Providers - Supabase Storage

```
create server s3_server
  foreign data wrapper s3_wrapper
  options (
    aws_access_key_id '<supabase_storage_access_key>',
    aws_secret_access_key '<supabase_storage_secret_key>',
    aws_region 'eu-central-1',
    endpoint_url 'https://<project_ref>.supabase.co/storage/v1/s3',
    path_style_url 'true'
  );
```

Connecting to S3-compliant Providers - Wasabi

```
create server s3_server
  foreign data wrapper s3_wrapper
  options (
    aws_access_key_id '<your_wasabi_access_key>',
    aws_secret_access_key '<your_wasabi_secret_key>',
    aws_region 'eu-central-1',
```

```
      endpoint_url 'https://s3.eu-central-1.wasabisys.com'
    );
```

Create a schema

We recommend creating a schema to hold all the foreign tables:

```
create schema if not exists s3;
```

Options

The following options are available when creating S3 foreign tables:

- `uri` - S3 URI, required. For example, `s3://bucket/s3_table.csv`
- `format` - File format, required. `csv`, `jsonl`, or `parquet`
- `has_header` - If the CSV file has header, optional. `true` or `false`, default is `false`
- `compress` - Compression algorithm, optional. One of `gzip`, `bzip2`, `xz`, `zlib`, default is no compression

Entities

CSV Files

This is an object representing CSV files in S3.

Ref: AWS S3 docs

Operations

Object	Select	Insert	Update	Delete	Truncate
CSV	✅	❌	❌	❌	❌

Usage

```
create foreign table s3.table_csv (
  name text,
  sex text,
  age text,
  height text,
  weight text
)
  server s3_server
  options (
    uri 's3://bucket/s3_table.csv',
    format 'csv',
    has_header 'true'
  );
```

Notes

- All columns must be defined in the foreign table
- All column types must be `text`
- Optional header support via `has_header` option
- Supports compression (gzip, bzip2, xz, zlib)

JSON Lines Files

This is an object representing JSONL files in S3.

Ref: JSONL docs

Operations

Object	Select	Insert	Update	Delete	Truncate
JSONL	✅	❌	❌	❌	❌

Usage

```
create foreign table s3.table_jsonl (
  name text,
  sex text,
  age text,
  height text,
  weight text
)
  server s3_server
  options (
    uri 's3://bucket/s3_table.jsonl',
    format 'jsonl'
  );
```

Notes

All columns must be defined in the foreign table

All column types must be `text`

Each line must be a valid JSON object

Supports compression (gzip, bzip2, xz, zlib)

Parquet Files

This is an object representing Parquet files in S3.

Ref: Parquet docs

Operations

Object	Select	Insert	Update	Delete	Truncate
Parquet	✅	❌	❌	❌	❌

Usage

```
create foreign table s3.table_parquet (
  id integer,
  bool_col boolean,
  bigint_col bigint,
  float_col real,
  date_string_col text,
  timestamp_col timestamp
)
  server s3_server
  options (
    uri 's3://bucket/s3_table.parquet',
    format 'parquet'
  );
```

Notes

- Not all columns need to be defined in foreign table
- Column names must match between Parquet file and foreign table
- Supports various Postgres data types (see Data Types section)
- Compressed files are loaded entirely into memory
- Supports compression (gzip, bzip2, xz, zlib)

Query Pushdown Support

This FDW doesn't support query pushdown.

Supported Data Types For Parquet File

The S3 Wrapper uses Parquet file data types from arrow_array::types, below are their mappings to Postgres data types.

Postgres Type	Parquet Type
boolean	BooleanType
char	Int8Type
smallint	Int16Type
real	Float32Type
integer	Int32Type
double precision	Float64Type
bigint	Int64Type
numeric	Float64Type
text	ByteArrayType
date	Date64Type
timestamp	TimestampNanosecondType
timestamptz	TimestampNanosecondType

Limitations

This section describes important limitations and considerations when using this FDW:

- Large result sets experience slower performance due to full data transfer requirement
- Compressed files are loaded entirely into memory and have additional processing overhead
- CSV and JSONL columns must be defined as `text` type only
- All columns must be defined in foreign tables for CSV and JSONL
- Column names must match exactly for Parquet files
- No support for S3 Select or other S3-side filtering

Materialized views using these foreign tables may fail during logical backups

Examples

Basic Example

This will create some "foreign table" inside your Postgres database can read data from S3:

```
-- CSV file, no compression
create foreign table s3.table_csv (
  name text,
  sex text,
  age text,
  height text,
  weight text
)
  server s3_server
  options (
    uri 's3://bucket/s3_table.csv',
    format 'csv',
    has_header 'true'
  );

-- JSON line file, no compression
create foreign table s3.table_jsonl (
  name text,
  sex text,
  age text,
  height text,
  weight text
)
  server s3_server
  options (
    uri 's3://bucket/s3_table.jsonl',
    format 'jsonl'
  );

-- GZIP compressed CSV file
create foreign table s3.table_csv_gzip (
  name text,
  sex text,
  age text,
```

```
    height text,
    weight text
  )
    server s3_server
    options (
      uri 's3://bucket/s3_table.csv.gz',
      format 'csv',
      has_header 'true',
      compress 'gzip'
    );

  -- Parquet file, no compression
  create foreign table s3.table_parquet (
    id integer,
    bool_col boolean,
    bigint_col bigint,
    float_col real,
    date_string_col text,
    timestamp_col timestamp
  )
    server s3_server
    options (
      uri 's3://bucket/s3_table.parquet',
      format 'parquet'
    );

  -- GZIP compressed Parquet file
  create foreign table s3.table_parquet_gz (
    id integer,
    bool_col boolean,
    bigint_col bigint,
    float_col real,
    date_string_col text,
    timestamp_col timestamp
  )
    server s3_server
    options (
      uri 's3://bucket/s3_table.parquet.gz',
      format 'parquet',
      compress 'gzip'
    );
```

Read from Supabase Storage

This example will read a CSV file stored on Supabase Storage. The access information can be found on Supabase Storage settings page.

```
create server s3_server
  foreign data wrapper s3_wrapper
  options (
    aws_access_key_id '<access key ID>',
    aws_secret_access_key '<secret access key>',
    aws_region '<region>',
    endpoint_url 'https://<project_ref>.supabase.co/storage/v1/s3',
    path_style_url 'true'
  );

create foreign table s3.supabase_table_csv (
  id text,
  name text
)
  server s3_server
  options (
    uri 's3://mybucket/myfile.csv',
    format 'csv',
    has_header 'true'
  );
```

Edit this page on GitHub

Need some help? Contact support

Latest product updates? See Changelog

Something's not right? Check system status

© Supabase Inc

—

Contributing

Author Styleguide

Open Source

SupaSquad

Terraform Provider

The Supabase Provider allows Terraform to manage resources hosted on Supabase platform.

You may use this provider to version control your project settings or setup CI/CD pipelines for automatically provisioning projects and branches.

- CI/CD example
- Step-by-step tutorials
- Contributing guide

Using the provider

This simple example imports an existing Supabase project and synchronises its API settings.

```
terraform {
  required_providers {
    supabase = {
      source  = "supabase/supabase"
      version = "~> 1.0"
    }
  }
}

provider "supabase" {
  access_token = file("${path.module}/access-token")
}

# Define a linked project variable as user input
variable "linked_project" {
  type = string
```

```
17    }
18
19    # Import the linked project resource
20    import {
21      to = supabase_project.production
22      id = var.linked_project
23    }
24
25    resource "supabase_project" "production" {
26      organization_id   = "nknnyrtlhxudbsbuazsu"
27      name              = "tf-project"
28      database_password = "tf-example"
29      region            = "ap-southeast-1"
30
31      lifecycle {
32        ignore_changes = [database_password]
33      }
34    }
35
36    # Configure api settings for the linked project
37    resource "supabase_settings" "production" {
38      project_ref = var.linked_project
39
40      api = jsonencode({
41        db_schema            = "public,storage,graphql_public"
42        db_extra_search_path = "public,extensions"
43        max_rows             = 1000
44      })
45    }
```

Edit this page on GitHub

Need some help? Contact support

Latest product updates? See Changelog

Something's not right? Check system status

© Supabase Inc

—

Contributing

Author Styleguide